Lab 1 — Getting hands-on with images

Author: Diaeddin Rimawi

What you will learn

By the end of this lab, you should be able to:

- Set up a clean Python environment for computer vision labs (folders + virtual environment).
- Run Jupyter (Notebook or JupyterLab) using your lab environment.
- Load and display images with **Pillow** and with **OpenCV**.
- Convert color spaces (RGB ↔ BGR, color ↔ grayscale).
- Inspect pixel values and explain what you see.
- Apply simple geometric transformations.
- Draw basic shapes on images.

---

0) Prerequisites (do this once)

Install Python (if you don't have it)

- Official download page: Python downloads (python.org)

> Tip (Windows): during installation, enable **"Add Python to PATH"**.

---

1) Organize your lab workspace (recommended structure)

Create a folder for the course labs, a single virtual environment for the course, and one folder per lab.

1.1 Create folders

In a terminal (macOS/Linux) or PowerShell (Windows):

```
mkdir -p cv-course-labs
cd cv-course-labs
mkdir -p ./data ./outputs
```

Suggested structure:

```
cv-course-labs/
  .venv/            # one virtual environment for the whole course
  Lab1/
```

```
    data/           # input images
    outputs/        # results you create
    notes.md        # your short answers / observations
```

Your turn: create the folder `./outputs/` and confirm it exists.

---

2) Create and activate a virtual environment

Why? A *virtual environment* isolates packages for this course so your system Python stays clean.

- Official Python docs: venv — Creation of virtual environments

2.1 Create the environment

From inside `cv-course-labs/`:

```
python -m venv .venv
```

2.2 Activate the environment

macOS / Linux:

```
source .venv/bin/activate
```

Windows (PowerShell):

```
.\.venv\Scripts\Activate
```

2.3 Quick check

```
python --version
python -m pip --version
```

---

3) Install the required libraries

We will install:

- **NumPy**: fast arrays + math (core tool for representing images as matrices).
- **Pillow**: simple image I/O and basic image processing (Python-friendly).

- **OpenCV**: computer vision toolkit (very feature-rich, fast, widely used).

3.1 pip basics

- pip docs: pip documentation

Upgrade pip (recommended):

```
python -m pip install --upgrade pip
```

You can jump to point 4, if you like to run the following using Jupyter.

3.2 Install NumPy

- What it is: n-dimensional arrays + vectorized operations.
- Docs + install guide: NumPy — Installing NumPy

Install:

```
python -m pip install numpy
```

Common usage examples (you will use these constantly):

```
import numpy as np

# 1) create an array
x = np.array([1, 2, 3])
print(x)

# 2) basic math (vectorized)
y = x * 10
print(y)

# 3) matricies
# A = np.zeros((2, 2), dtype=np.uint8)
# A = np.array([[1, 2], [3, 4]])
A = np.array([1, 2])

print(A)
# Rotation Matrix
R = np.array([[0, -1], [1, 0]])
print(R)
# Dot product
B = np.dot(R, A)
print(B)
```

```
R_invers = np.array([[0, 1], [-1, 0]])
B_prime = np.dot(R_invers, B)
print(B_prime)
# Shape function
print(A.shape)
```

3.3 Install Pillow

- What it is: a maintained fork of PIL (Python Imaging Library) for image loading, saving, and simple transforms.
- Install docs: Pillow — Basic installation
- Tutorial (good starting point): Pillow Handbook — Tutorial

Install:

```
python -m pip install pillow
```

You can use your own image, or download some images and store into data folder (see point 5)

Common usage examples, don't forget to create outputs folder:

```
from PIL import Image

img = Image.open("./data/lenna.png")
print(img.size, img.mode)  # (width, height), and mode like 'RGB'

gray = img.convert("L")
gray.save("./outputs/lenna_gray_pillow.png")

# Following line to view image in Jupyter
gray
```

Try threshold using PIL

```
threshold = 128
img_thresholded = gray.point(lambda p: 255 if p > threshold else 0)
```

Can you explain what happened?

3.4 Install OpenCV

- What it is: a full computer vision library (filters, features, geometry, calibration, video, etc.).
- Official OpenCV pip quick-start: OpenCV — Install OpenCV for Python with pip
- OpenCV-Python tutorials: OpenCV-Python Tutorials

Install:

```
python -m pip install opencv-python
```

Common usage examples:

```
import cv2

img_bgr = cv2.imread("./data/lenna.png")
print(img_bgr.shape)  # (height, width, channels)

img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
cv2.imwrite("./outputs/lenna_gray_opencv.png", img_gray)
```

3.5 Verify your installs

```
python -c "import numpy as np; import PIL; import cv2; print('OK', np.__version__,
PIL.__version__, cv2.__version__)"
```

---

4) Install and run Jupyter

You can use either **Jupyter Notebook** or **JupyterLab**.

- Official install page: Project Jupyter — Installing Jupyter
- Jupyter documentation hub: Project Jupyter Documentation
- "What is the Jupyter Notebook?": Jupyter Notebook docs

4.1 Install Jupyter in your virtual environment

From `cv-course-labs/` with the environment activated:

```
python -m pip install notebook jupyterlab
```

4.2 Start Jupyter

```
jupyter lab
```

(or)

```
jupyter notebook
```

> You can do this lab either in a notebook **or** by creating a normal Python file `./lab01.py`. The instructions below give Python code blocks you can copy/paste and run.

Your turn: start JupyterLab and create a file `./notes.md` where you will answer the "describe what you see" questions.

---

5) Download sample images

We'll use three classic test images.

Put them in `./data/`.

Option A — in a Jupyter cell (the `!` is Jupyter syntax)

```
!wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-CV0101EN-SkillsNetwork/images%20/images_part_1/lenna.png -O ./data/lenna.png
!wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-CV0101EN-SkillsNetwork/images%20/images_part_1/baboon.png -O ./data/baboon.png
!wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-CV0101EN-SkillsNetwork/images%20/images_part_1/barbara.png -O ./data/barbara.png
```

Option B — in a terminal (macOS/Linux)

```
wget "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-CV0101EN-SkillsNetwork/images%20/images_part_1/lenna.png" -O ./data/lenna.png
wget "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-CV0101EN-SkillsNetwork/images%20/images_part_1/baboon.png" -O ./data/baboon.png
wget "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-CV0101EN-SkillsNetwork/images%20/images_part_1/barbara.png" -O ./data/barbara.png
```

Option C — if you don't have `wget` (PowerShell)

```
Invoke-WebRequest -Uri "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-CV0101EN-SkillsNetwork/images%20/images_part_1/lenna.png" -OutFile "./data/lenna.png"
Invoke-WebRequest -Uri "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-CV0101EN-SkillsNetwork/images%20/images_part_1/baboon.png" -OutFile "./data/baboon.png"
Invoke-WebRequest -Uri "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-CV0101EN-SkillsNetwork/images%20/images_part_1/barbara.png" -OutFile "./data/barbara.png"
```

Checkpoint: confirm the files exist by listing the folder (`ls ./data` or on `Windows` > `dir .\data`).

Part A — Warm-up with NumPy (no Pillow/OpenCV yet)

A1) Create a tiny black/white "image" as a matrix

Images can be represented as matrices. For a grayscale image, each pixel is a single number.

Task: create an 8×8 checkerboard (0 = black, 255 = white) and print it.

```
import numpy as np

# 8x8 checkerboard: alternating 0 and 255
img0 = np.indices((8, 8)).sum(axis=0) % 2
img0 = (img0 * 255).astype(np.uint8)

print("Original 8x8 checkerboard:")
print(img0)
```

Describe what you see:

- How many unique values exist?
- What is the pattern?

Your turn (no code shown): create a *vertical stripes* pattern instead of a checkerboard.

A2) Transformation 1 — scaling (upsampling)

We will "scale" by a factor of 2 using a simple nearest-neighbor idea: repeat pixels.

```
# Reset to original each time
img = img0.copy()

scaled2 = np.kron(img, np.ones((2, 2), dtype=np.uint8))
print("\nScaled (x2) shape:", scaled2.shape)
print(scaled2)
```

Describe what you see:

- What changed in the numbers?
- What changed in the size?

Your turn: try scaling by 3 instead of 2.

A3) Transformation 2 — rotation by 90 degrees

Rotation by 90° can be done by swapping axes.

```
img = img0.copy()

rot90 = np.rot90(img, k=1)  # 90 degrees counter-clockwise
print("\nRotated 90°:")
print(rot90)
```

Describe what you see:

- Is the checkerboard different after 90° rotation? Why/why not?

Your turn: rotate by 180° using NumPy.

Part B — Tasks with real images

Helper note: RGB vs BGR (important!)

- **Pillow** usually uses **RGB** order.
- **OpenCV (cv2)** reads color images as **BGR** by default.

This difference often explains “weird colors” when you switch between them.

Task 1 — Read images (Pillow and OpenCV)

1A) Read with Pillow

```
from PIL import Image

img_pil = Image.open("./data/lenna.png")
print("Pillow:")
print("  size (W,H):", img_pil.size)
print("  mode:", img_pil.mode)

# Optional display (opens default viewer)
img_pil.show()
```

1B) Read with OpenCV

```
import cv2

img_cv = cv2.imread("./data/lenna.png")  # BGR
print("OpenCV:")
print("  shape (H,W,C):", img_cv.shape)
print("  dtype:", img_cv.dtype)
cv2.imshow("Lenna (OpenCV)", img_cv)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Describe what you see:

- Why does Pillow report size as (W,H) but OpenCV shape is (H,W,C)?

Your turn: load `baboon.png` with both libraries and print its size/shape.

Task 2 — Display images (and save outputs)

Create an output folder if you didn't:

```
import os
os.makedirs("./outputs", exist_ok=True)
```

2A) Display / save with Pillow

```
# Save a copy to confirm your pipeline works
img_pil.save("./outputs/lenna_copy_pillow.png")
```

```
# Display (may open a viewer window)
img_pil.show(title="Lenna (Pillow)")
```

2B) Display / save with OpenCV

```
cv2.imwrite("./outputs/lenna_copy_opencv.png", img_cv)

# Display (requires a GUI)
cv2.imshow("Lenna (OpenCV)", img_cv)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Describe what you see:

- Are the saved images identical?
- If you display with OpenCV, do the colors look correct? (Hint: BGR)

Your turn: save `barbara.png` to outputs with both libraries.

Task 3 — Convert color (RGB/BGR and grayscale)

3A) Pillow: convert to grayscale

```
gray_pil = img_pil.convert("L")
print("Pillow grayscale mode:", gray_pil.mode)

gray_pil.save("./outputs/lenna_gray_pillow.png")
```

3B) OpenCV: BGR → grayscale

```
gray_cv = cv2.cvtColor(img_cv, cv2.COLOR_BGR2GRAY)
print("OpenCV grayscale shape:", gray_cv.shape)

cv2.imwrite("./outputs/lenna_gray_opencv.png", gray_cv)
```

3C) OpenCV: BGR → RGB (useful for correct colors)

```
rgb_cv = cv2.cvtColor(img_cv, cv2.COLOR_BGR2RGB)
cv2.imwrite("./outputs/lenna_rgb_converted_from_opencv.png", rgb_cv)
```

Describe what you see:

- What is the difference between a color image array and a grayscale array?

Your turn: convert `baboon.png` to grayscale with both libraries.

Task 4 — Inspect pixel values

4A) Pillow pixel inspection

```
# getpixel uses (x, y)
px00 = img_pil.getpixel((0, 0))
px10 = img_pil.getpixel((1, 0))
print("Pillow pixel (0,0):", px00)
print("Pillow pixel (1,0):", px10)
```

4B) OpenCV pixel inspection

```
# OpenCV array indexing is [y, x] and color is BGR
px00_cv = img_cv[0, 0]
px10_cv = img_cv[0, 1]
print("OpenCV pixel [0,0] (BGR):", px00_cv)
print("OpenCV pixel [0,1] (BGR):", px10_cv)
```

4C) Convert Pillow to NumPy and compare

```
import numpy as np

arr_pil = np.array(img_pil)  # RGB
print("NumPy from Pillow shape:", arr_pil.shape)
print("arr_pil[0,0] (RGB):", arr_pil[0, 0])
```

Describe what you see:

- Explain why `img_pil.getpixel((x,y))` vs `img_cv[y,x]` looks “swapped”.
- Explain why the color channel order differs.

Your turn: pick a pixel near the center and compare its values in Pillow vs OpenCV.

Task 5 — Apply simple transformations

We will do: resize (scaling), rotate, and flip.

5A) Pillow transformations

```
# Resize (scale down)
small_pil = img_pil.resize((img_pil.size[0] // 2, img_pil.size[1] // 2))
small_pil.save("./outputs/lenna_small_pillow.png")

# Rotate (degrees, around image center)
rot_pil = img_pil.rotate(30)  # default expands=False
rot_pil.save("./outputs/lenna_rot30_pillow.png")

# Flip left-right
flip_pil = img_pil.transpose(Image.FLIP_LEFT_RIGHT)
flip_pil.save("./outputs/lenna_flip_lr_pillow.png")
```

5B) OpenCV transformations

```
h, w = img_cv.shape[:2]

# Resize
small_cv = cv2.resize(img_cv, (w // 2, h // 2))
cv2.imwrite("./outputs/lenna_small_opencv.png", small_cv)

# Rotate 90 degrees (built-in)
rot90_cv = cv2.rotate(img_cv, cv2.ROTATE_90_COUNTERCLOCKWISE)
cv2.imwrite("./outputs/lenna_rot90_opencv.png", rot90_cv)

# Flip: 1=horizontal, 0=vertical, -1=both
flip_cv = cv2.flip(img_cv, 1)
cv2.imwrite("./outputs/lenna_flip_lr_opencv.png", flip_cv)
```

Describe what you see:

- Which library was easier for rotation?

Your turn: try a different rotation angle in OpenCV (hint: `cv2.getRotationMatrix2D` + `cv2.warpAffine`). Search it quickly in the official OpenCV tutorials.

Task 6 — Draw geometric shapes

6A) Pillow: draw on an image

- Pillow drawing reference (starter): Pillow — ImageDraw (If this page doesn't load on your device, use the Pillow docs search for "ImageDraw".)

```
from PIL import ImageDraw, Image

img = Image.open("./data/lenna.png")
img_draw_pil = img.copy()
draw = ImageDraw.Draw(img_draw_pil)

# Rectangle: [(x0,y0), (x1,y1)]
draw.rectangle([(50, 50), (200, 200)], outline="red", width=3)

# Line: [(x0,y0), (x1,y1)]
draw.line([(0, 0), (img_draw_pil.size[0]-1, img_draw_pil.size[1]-1)], fill="yellow",
width=3)

# Text
draw.text((60, 210), "Lab 1", fill="white")

img_draw_pil.save("./outputs/lenna_draw_pillow.png")
img_draw_pil.show()
```

6B) OpenCV: draw on an image

- Official tutorial: OpenCV — Drawing Functions in OpenCV

```
img_draw_cv = img_cv.copy()

# Rectangle: top-left, bottom-right
cv2.rectangle(img_draw_cv, (50, 50), (200, 200), (0, 0, 255), 3)  # red in BGR

# Line
h, w = img_draw_cv.shape[:2]
cv2.line(img_draw_cv, (0, 0), (w-1, h-1), (0, 255, 255), 3)  # yellow in BGR

# Text
cv2.putText(img_draw_cv, "Lab 1", (60, 230), cv2.FONT_HERSHEY_SIMPLEX, 1.0, (255,
255, 255), 2)

cv2.imwrite("./outputs/lenna_draw_opencv.png", img_draw_cv)
```

Describe what you see:

- Why do we write colors as (B, G, R) tuples in OpenCV but use names like "red" in Pillow?

Your turn: draw a circle around the center of the image using both libraries.